



نام درس: شبکه های کامپیوتری
نویسنده: محمد امانلو - ۸۱۰۱۰۰۰۸۴



تکلیف شماره ۴

سؤال ۱: مسیریابی با طولانی ترین تطابق پیشوند

فرض کنید روتر در یک شبکه دیتاگرامی با آدرس های 8-bit کار می کند و برای ارسال بسته ها از قاعده Longest Prefix Matching استفاده می کند. جدول ارسال روتر به صورت زیر است:

<i>Interface</i>	<i>PrefixMatch</i>
۱	۱۰
۲	۱۱
۳	۱۰۰
۴	۱۱۱
۵	۰۰۰
۶	otherwise

۱.۱) پاسخ تشریحی

(الف) اگر دو ورودی دقیقاً هم طول باشند و هر دو با یک آدرس مقصد تطابق داشته باشند، روتر چگونه انتخاب می کند؟
(با مثال)

در قاعده Longest Prefix Matching اصل انتخاب این است که «طولانی ترین پیشوند منطبق» انتخاب شود. اما اگر دو ورودی هم طول باشند و هر دو با یک آدرس مقصد منطبق شوند، در عمل دو حالت داریم:

حالت ۱: دو پیشوند هم طول ولی متفاوت اگر دو پیشوند هم طول متفاوت باشند، نمی توانند هم زمان با یک آدرس ثابت منطبق شوند. چون انطباق پیشوند یعنی چند بیت اول مقصد دقیقاً برابر پیشوند باشد؛ اگر دو پیشوند هم طول باشند و هر دو با مقصد منطبق شوند، آن چند بیت اول مقصد باید هم زمان برابر هر دو پیشوند باشد؛ پس آن دو پیشوند ناچاراً باید یکسان باشند. بنابراین «دو پیشوند متفاوت هم طول که هر دو با یک مقصد منطبق شوند» از نظر منطقی رخ نمی دهد.

حالت ۲: دو ورودی تکراری با پیشوند یکسان و طول یکسان (مسیرهای هم‌ارزش) آنچه در دنیای واقعی ممکن است رخ دهد این است که دو مسیر مختلف با یک پیشوند دقیقاً یکسان (و در نتیجه طول یکسان) در جدول وجود داشته باشند (مثلاً از دو پروتکل یا دو مسیر جایگزین)، مثل ECMP یا تداخل اطلاعات پروتکل‌ها. در این حالت، معیار انتخاب معمولاً در صفحه کنترل (نه صرفاً در صفحه داده) تعیین می‌شود، از جمله:

□ ترجیح پروتکل/منبع مسیر (مثلاً Administrative Distance پایین‌تر)

□ هزینه/متریک کمتر (مثلاً IGP metric)

□ اگر هزینه برابر باشد: ECMP و تقسیم بار (مثلاً per-flow hashing)

□ یا در ساده‌ترین پیاده‌سازی‌ها: انتخاب اولین ورودی جدول (وابسته به پیاده‌سازی)

مثال ساده فرض کنید دو ورودی زیر (با پیشوند کاملاً یکسان) در جدول وجود داشته باشد:

$$(\text{Interface}^1 \rightarrow 10), \quad (\text{Interface}^7 \rightarrow 10)$$

و آدرس مقصد ۱۰۱۰۱۱۱۰ باشد. هر دو مسیر با پیشوند ۱۰ (طول ۲) منطبق‌اند. روتر ممکن است:

□ مسیر با متریک کمتر را انتخاب کند، یا

□ اگر هر دو برابر بودند، از ECMP استفاده کند و برخی جریان‌ها را از Interface^۱ و برخی را از Interface^۷ عبور دهد.

(ب) اگر سطر otherwise (مسیر default route) حذف شود، برای بسته‌های بدون تطابق چه می‌شود؟

اگر ورودی otherwise (که نقش default route را دارد) حذف شود، برای هر بسته‌ای که با هیچ‌یک از پیشوندهای جدول منطبق نشود، روتر هیچ مسیر خروجی در Forwarding Table پیدا نمی‌کند. نتیجه عملی:

۱. بسته □□□□ می‌شود: روتر بسته را دور می‌اندازد چون نمی‌داند آن را به کدام next-hop یا interface بفرستد.

۲. ارسال پیام خطا در لایه شبکه:

□ در IPv4 معمولاً پیام ICMP Destination Unreachable (مثلاً Network Unreachable یا No route

to host) به مبدأ ارسال می‌شود (اگر سیاست‌ها اجازه دهد).

□ در IPv6 مشابه آن ICMPv6 Destination Unreachable ارسال می‌شود.

۳. اثر عملی در شبکه: حذف default route باعث می شود هر مقصدی که در جدول به طور صریح (با یک پیشوند مناسب) پوشش داده نشده، غیرقابل دسترس شود؛ در اینترنت واقعی، وجود مسیر پیش فرض برای بسیاری از روترها/شبکه های لبه ضروری است تا ترافیک ناشناخته به سمت مسیر بالادستی (upstream) هدایت شود.

۲.۱) تعیین Interface برای آدرس های داده شده با قاعده Longest Prefix Match

قاعده: برای هر آدرس مقصد، همه پیشوندهایی که با بیت های ابتدایی آدرس منطبق می شوند را پیدا می کنیم؛ سپس پیشوند با بیشترین طول انتخاب می شود.

(الف) ۰۰۰۰۰۰۱۱

□ بررسی پیشوند ۰۰۰: سه بیت اول مقصد ۰۰۰ است، پس منطبق است (طول ۳).

□ سایر پیشوندها مثل ۱۰، ۱۱، ۱۰۰، ۱۱۱ منطبق نیستند.

پس طولانی ترین پیشوند منطبق ۰۰۰ است $\Rightarrow$ ارسال به:

Interface ۵

(ب) ۱۰۱۰۱۱۱۰

□ پیشوند ۱۰: دو بیت اول مقصد ۱۰ است، پس منطبق است (طول ۲). $\Rightarrow$ Interface 1

□ پیشوند ۱۰۰: سه بیت اول مقصد ۱۰۱ است، پس منطبق نیست.

□ ۱۱ و ۱۱۱ و ۰۰۰ نیز منطبق نیستند.

پس طولانی ترین پیشوند منطبق ۱۰ است $\Rightarrow$ ارسال به:

Interface ۱

(ج) ۱

از آنجا که در صورت سؤال گفته شده آدرس ها 8-bit هستند، نوشتن ۱ به احتمال زیاد یعنی «عدد ۱» که در نمایش دودویی 8-bit برابر است با:

$$1_2 = 00000001_2$$

اکنون تطبیق پیشوند:

□ سه بیت اول ۰۰۰ است، پس پیشوند ۰۰۰ منطبق می شود (طول ۳) $\Rightarrow$ Interface 5.

بنابراین:

Interface ۵

نکته تکمیلی (اگر منظور «آدرس هایی که با بیت ۱ شروع می شوند» باشد) اگر واقعاً منظور از ۱ این باشد که فقط بیت اول ۱ است (یعنی ۱xxxxxxx)، آنگاه پاسخ یکتا نیست و به بیت های بعدی بستگی دارد:

<i>Addressstart</i>	<i>Interface(longest – prefixmatch)</i>
۱۰۰xxx	۳
۱۱۱xxx	۴
۱۰xxxxxx(butnot ۱۰۰)	۱
۱۱xxxxxx(butnot ۱۱۱)	۲

اما با تفسیر رایج «۱ یعنی مقدار ۱ در 8-bit»، نتیجه همان ۰۰۰۰۰۰۰۱ و Interface 5 است.

سؤال ۲: زمان بندی بسته ها در روتر (Packet Scheduling) با WFQ

فرض: زمان slotted است و هر اسلات در لحظات صحیح $t = ۰, ۱, ۲, \dots$ شروع می شود. بسته ها می توانند در هر لحظه ای داخل یک اسلات وارد شوند؛ در ابتدای هر اسلات، scheduler دقیقاً یکی از بسته های موجود در صف را برای ارسال انتخاب می کند و ارسال آن بسته دقیقاً یک اسلات طول می کشد (اگر در ابتدای اسلات t انتخاب شود، در انتهای اسلات و زمان $t + ۱$ تمام می شود).

الگوریتم زمان بندی: WFQ (مخفف Weighted Fair Queueing).

سه کلاس ترافیک داریم: (۱, ۲, ۳) که عدد کوچک تر یعنی اولویت بالاتر (از نگاه سرویس دهی مطلوب). وزن ها در WFQ به ترتیب:

$$w_1 = ۰,۵, \quad w_2 = ۰,۳, \quad w_3 = ۰,۲.$$

(در WFQ «اولویت» به صورت سخت گیرانه مثل Priority Queueing اعمال نمی شود، بلکه «سهم سرویس» متناسب با وزن ها تضمین می شود).

۱.۲) پاسخ تشریحی

(الف) ایده اصلی WFQ و اینکه چرا از Priority Queueing منصفانه تر است

ایده اصلی WFQ تلاش می کند رفتار ایده آل GPS (مخفف Generalized Processor Sharing) را به صورت بسته ای

تقریب بزند. در GPS لینک به صورت پیوسته بین کلاس ها/جریان های دارای بک لاگ تقسیم می شود و هر کلاس i در هر لحظه ای که در صف بسته دارد، نرخ سرویس

$$r_i(t) = \frac{w_i}{\sum_{j \in B(t)} w_j} C$$

می گیرد که در آن:

□ C ظرفیت لینک (اینجا در هر اسلات یک بسته، یعنی نرخ مؤثر ثابت)

□ $B(t)$ مجموعه کلاس های backlogged در زمان t (کلاس هایی که صفشان خالی نیست)

اما چون در واقعیت سرویس دهی بسته ای است (نمی شود یک بسته را «نیمه» ارسال کرد)، WFQ با مفهوم «زمان مجازی» و «برچسب» (tag) برای هر بسته کاری می کند که ترتیب سرویس دهی تا حد ممکن شبیه GPS شود.

برچسب های WFQ برای بسته k ام از کلاس i با زمان ورود a_i^k و طول L_i^k (در این سؤال همه طول یکسان دارند و هر بسته یک اسلات می خواهد، پس معمولاً $L = 1$)، دو برچسب تعریف می شود:

$$S_i^k = \max \left(F_i^{k-1}, V(a_i^k) \right)$$

$$F_i^k = S_i^k + \frac{L_i^k}{w_i}$$

که در آن:

□ $V(t)$ «زمان مجازی» (virtual time) سیستم است؛ به طور شهودی شبیه ساعت GPS جلو می رود.

□ F_i^{k-1} پایان مجازی بسته قبلی همان کلاس است (برای حفظ ترتیب درون کلاسی).

قانون انتخاب در هر لحظه زمان بندی (ابتدای هر اسلات)، از بین بسته های موجود در صف، بسته ای انتخاب می شود که کمترین F را دارد (کوچک ترین virtual finish time).

چرا از Priority Queueing منصفانه تر است؟ در Priority Queueing (اولویت سخت گیرانه)، اگر کلاس های با اولویت بالا دائماً بسته داشته باشند، کلاس های پایین ممکن است هیچ وقت سرویس نگیرند (یعنی starvation). اما در WFQ حتی اگر کلاس ۱ همیشه شلوغ باشد، کلاس ۲ و ۳ به نسبت وزنشان سهم سرویس می گیرند؛ یعنی حق سرویس (حداقل سهم) برای کلاس های پایین تضمین می شود و عدالت وزنی برقرار است:

$$i \text{ سهم بلنمدت کلاس } \approx \frac{w_i}{\sum_j w_j}.$$

در این سؤال اگر هر سه کلاس همیشه بک لاگ باشند:

$$\sum w = 0.5 + 0.3 + 0.2 = 1 \Rightarrow (50\%, 30\%, 20\%) \text{ سهم‌ها}$$

(ب) آیا با این وزن‌ها امکان starvation برای کلاس ۳ وجود دارد؟ مقایسه با Priority Queueing

پاسخ کوتاه در WFQ با وزن‌های مثبت (اینجا $w_3 = 0.2 > 0$)، starvation رخ نمی‌دهد. در Priority Queueing (اولویت سخت‌گیرانه)، starvation برای کلاس ۳ ممکن است.

استدلال دقیق برای WFQ فرض کنید کلاس ۳ در یک بازه طولانی همیشه بسته دارد (یعنی backlogged است). در بدترین حالت، کلاس‌های ۱ و ۲ هم همیشه بسته دارند. در این صورت در مدل GPS کلاس ۳ نرخ سرویس حداقلی:

$$r_3 = \frac{w_3}{w_1 + w_2 + w_3} C = \frac{0.2}{1} C = 0.2C$$

می‌گیرد. WFQ تقریب بسته‌ای همین رفتار است؛ بنابراین کلاس ۳ به صورت بلندمدت حداقل حدود ۲۰٪ ظرفیت را می‌گیرد. پس:

□ ممکن است تاخیر کلاس ۳ نسبت به کلاس ۱ بیشتر باشد،

□ اما «کاملاً بی سرویس ماندن» (یعنی starvation) اتفاق نمی‌افتد.

مقایسه با Priority Queueing در Priority Queueing اگر کلاس ۱ و/یا ۲ دائماً بسته باشند، زمان‌بند همیشه از همان‌ها انتخاب می‌کند و کلاس ۳ ممکن است برای همیشه در صف بماند:

کلاس ۳ ممکن است هرگز سرویس نگیرد. $\Rightarrow$ اگر ترافیک کلاس‌های با اولویت بالاتر اشباع باشد

پس:

□ WFQ: عدالت وزنی و جلوگیری از starvation

□ Priority: امکان starvation برای کلاس‌های پایین

۲.۲) تعیین ترتیب ارسال بسته‌ها از $t = 1$ تا $t = 15$ با WFQ

نکته کلیدی برای اینکه «ترتیب دقیق ارسال بسته‌های شماره‌دار» را بنویسیم، باید داده‌های شکل (برای هر بسته: زمان ورود دقیق داخل اسلات + کلاس بسته) را داشته باشیم. در متن سؤال، شکل ارجاع داده شده ولی اینجا خود

شکل/داده‌هایش ارائه نشده است؛ بنابراین من نمی‌توانم بدون اطلاعات شکل، شماره دقیق بسته ارسالی در هر اسلات را به صورت عددی و قطعی استخراج کنم (و حدس زدن هم غلط و غیردقیق می‌شود).
با این حال، روش کامل و دقیق محاسبه را می‌دهم تا دقیقاً همان خروجی خواسته شده (جدول اسلات ۱ تا ۵) به دست بیاید؛ به محض اینکه لیست ورودها را داشته باشید، جدول نهایی مکانیکی و بدون ابهام پر می‌شود.

گام‌های محاسبه WFQ در این سؤال (کاملاً عملیاتی)

فرض‌های این سؤال

□ هر بسته دقیقاً یک اسلات برای ارسال می‌خواهد $L = 1 \Rightarrow$.

□ بسته‌ای که در طول اسلات $(t, t+1]$ وارد می‌شود، در ابتدای اسلات $t+1$ برای انتخاب در صف «قابل مشاهده» است (چون انتخاب در ابتدای اسلات انجام می‌شود).

□ اگر زمان‌های ورود داخل اسلات مهم باشند (طبق شکل)، باید $V(a)$ را در همان لحظه ورود حساب کنیم.

۱) نگه‌داشتن زمان مجازی $V(t)$ در GPS، زمان مجازی وقتی لینک مشغول است با نرخ:

$$\frac{dV(t)}{dt} = \frac{1}{\sum_{j \in \mathcal{B}(t)} w_j}$$

زیاد می‌شود. یعنی اگر در یک بازه زمانی Δt ، مجموعه بک‌لاگ ثابت باشد و مجموع وزن‌های بک‌لاگ $W = \sum w$ باشد:

$$V(t + \Delta t) = V(t) + \frac{\Delta t}{W}.$$

در حالت اسلاتی، اگر کل اسلات را مشغول ارسال باشیم و در آن اسلات مجموع وزن‌های بک‌لاگ (در آن بازه) برابر W باشد، افزایش V در آن اسلات تقریباً:

$$\Delta V = \frac{1}{W}.$$

۲) برای هر بسته برچسب‌ها را بسازید برای بسته شماره p که متعلق به کلاس i است و در زمان a_p وارد می‌شود:

$$S_p = \max(F, \text{آخرین بسته همان کلاس}, V(a_p))$$

$$F_p = S_p + \frac{1}{w_i}$$

چون $L = 1$.

۳) قانون انتخاب در ابتدای هر اسلات در ابتدای اسلات t ، از بین همه بسته‌هایی که تا آن لحظه رسیده‌اند و هنوز ارسال نشده‌اند، بسته‌ای را انتخاب کنید که کمترین F را دارد:

$$p^*(t) = \arg \min_{p \in \text{بسته‌های رسیده و ارسال نشده در } t} F_p.$$

اگر تساوی شد (دو بسته F برابر داشتند)، معمولاً با قانون شکستن تساوی مثل FIFO (قدیمی‌تر اول) یا شماره کوچک‌تر تصمیم می‌گیرند (وابسته به پیاده‌سازی؛ ولی باید یک tie-break ثابت داشته باشید).

۴) بروزرسانی صف و $V(t)$ بسته انتخاب‌شده کل اسلات $(t, t+1)$ را اشغال می‌کند؛ در پایان اسلات: □ آن بسته حذف می‌شود (ارسال شده است).

□ V به اندازه ΔV (بسته به مجموع وزن‌های بک‌لاگ در طول اسلات) جلو می‌رود.

نمایش خروجی موردنیاز سؤال (قالب جدول اسلات‌ها)

خروجی خواسته‌شده این است که برای $t = 1$ تا $t = 15$ بنویسیم در هر اسلات کدام «شماره بسته» ارسال می‌شود:

Slott	$(t, t+1)$ بسته ارسالی در
۱	
۲	
۳	
⋮	⋮
۱۵	

برای پر کردن این جدول باید از شکل، این داده‌ها را داشته باشیم:

$$\{(p, a_p, \text{class}(p))\}_{p=1}^{15}.$$

اگر شکل را دارید ولی می‌خواهید سریع به جواب برسید کافی است یک لیست ساده از این فرم استخراج کنید (حتی دستی):

$$p : \text{class} = i, \quad a_p = \text{مثلاً } 2, 3$$

و سپس دقیقاً با فرمول‌های بالا S, F را بسازید و در هر اسلات کمترین F را انتخاب کنید؛ خروجی دقیقاً همان چیزی می‌شود که سؤال می‌خواهد.

نکته کاربردی: سهم سرویس با وزن ها

برای درک سریع رفتار، اگر هر سه کلاس همزمان صف داشته باشند، الگوی سرویس دهی بلندمدت معمولاً نزدیک به:

$$۲ : ۳ : ۵ \approx \text{کلاس } ۱ : \text{کلاس } ۲ : \text{کلاس } ۳$$

است (چون وزن ها ۰,۲, ۰,۳, ۰,۵ هستند). ولی ترتیب دقیق بسته های شماره دار به لحظه ورود هر بسته و وضعیت صف در هر اسلات بستگی دارد.

سؤال ۳: Subnet Addressing

فضای آدرس داده شده:

۲۱۶,۱۳۵,۴۶,۰/۲۴

تعداد میزبان های مورد نیاز:

$$A : ۱۲۱ \quad B : ۴۹ \quad C : ۲۱$$

هدف: تقسیم آدرس ها بین سه زیر شبکه به گونه ای که برای هر زیر شبکه کمترین فضای ممکن (بدون کمبود آدرس) اختصاص داده شود و تا حد امکان یک فضای آدرس پیوسته برای زیر شبکه آینده آزاد بماند.

۱.۳) پاسخ های مفهومی

(الف) آیا 216.135.46.0/24 عمومی است یا خصوصی؟ چرا؟

این بلوک عمومی (Public) است؛ چون در بازه های آدرس خصوصی RFC1918 قرار نمی گیرد. بازه های خصوصی عبارتند از:

$$۱۰,۰,۰,۰/۸, \quad ۱۷۲,۱۶,۰,۰/۱۲, \quad ۱۹۲,۱۶۸,۰,۰/۱۶$$

در حالی که آدرس 216.135.46.0/24 خارج از این محدوده هاست، پس به صورت معمول قابل routable شدن در اینترنت بوده و خصوصی محسوب نمی شود.

(ب) در یک بلوک 24/ حداکثر چند میزبان داریم و چرا ۲ آدرس کم می شود؟

در 24/ تعداد بیت های میزبان برابر است با:

$$۳۲ - ۲۴ = ۸$$

پس تعداد کل آدرس های موجود در بلوک:

$$2^8 = 256$$

اما از این ۲۵۶ آدرس، دو آدرس قابل تخصیص به میزبان ها نیست:

□ آدرس شبکه (Network Address): حالتی که همه بیت های میزبان صفر باشند (برای شناسایی خود زیر شبکه استفاده می شود).

□ آدرس پخش همگانی (Broadcast Address): حالتی که همه بیت های میزبان یک باشند (برای ارسال به همه میزبان های همان زیر شبکه استفاده می شود).

بنابراین حداکثر تعداد میزبان های قابل استفاده در یک 24/ برابر است با:

$$256 - 2 = \boxed{254}$$

۲.۳) طراحی زیر شبکه ها (CIDR / VLSM)

برای کمترین اتلاف آدرس، برای هر زیر شبکه کوچک ترین اندازه بلوکی را انتخاب می کنیم که تعداد آدرس های قابل تخصیص آن $\geq$ تعداد میزبان مورد نیاز باشد. در هر زیر شبکه اگر طول پیشوند x باشد، تعداد آدرس های قابل استفاده:

$$2^{32-x} - 2$$

(الف) تعیین CIDR هر زیر شبکه بهینه + حفظ یک فضای پیوسته آزاد

زیر شبکه A (۱۲۱ میزبان) می خواهیم:

$$2^{32-x} - 2 \geq 121$$

اگر $2^{32-x} - 2 = 126 \Rightarrow 2^7 - 2 = 126 \Rightarrow 32 - x = 7$ که کافی است. پس:

$$\boxed{A: /25}$$

چون 25/ یعنی ۱۲۸ آدرس (۱۲۶ قابل استفاده).

زیر شبکه B (۴۹ میزبان) اگر $2^6 - 2 = 62 \Rightarrow 32 - x = 6$ که کافی است. پس:

$$\boxed{B: /26}$$

زیر شبکه C (۲۱ میزبان) اگر $۳۰ - ۲ = ۲۵ \Rightarrow ۳۲ - x = ۵$ که کافی است. پس:

$$C: /۲۷$$

چیدمان برای باقی ماندن یک فضای آزاد پیوسته کل بلوک 24/ شامل ۲۵۶ آدرس است. مجموع آدرس های مصرفی:

$$۱۲۸ + ۶۴ + ۳۲ = ۲۲۴$$

پس ۳۲ آدرس باقی می ماند. برای اینکه این ۳۲ آدرس پیوسته بماند، زیر شبکه ها را از ابتدای بلوک و به ترتیب بزرگ به کوچک پشت سر هم می چینیم:

$$\begin{aligned} A: & ۲۱۶,۱۳۵,۴۶,۰/۲۵ \quad (۰ \text{ تا } ۱۲۷) \\ B: & ۲۱۶,۱۳۵,۴۶,۱۲۸/۲۶ \quad (۱۲۸ \text{ تا } ۱۹۱) \\ C: & ۲۱۶,۱۳۵,۴۶,۱۹۲/۲۷ \quad (۱۹۲ \text{ تا } ۲۲۳) \\ & ۲۱۶,۱۳۵,۴۶,۲۲۴/۲۷ \quad (۲۲۴ \text{ تا } ۲۵۵) \text{ Free (برای آینده):} \end{aligned}$$

به این ترتیب یک بلوک آزاد کاملاً پیوسته برای اضافه کردن یک زیر شبکه جدید در آینده باقی می ماند.

(ب) مشخصات زیر شبکه: broadcast A، اولین و آخرین host

زیر شبکه:

$$A = ۲۱۶,۱۳۵,۴۶,۰/۲۵$$

در 25/ اندازه بلوک ۱۲۸ است، پس بازه آدرس ها:

$$۲۱۶,۱۳۵,۴۶,۰ \text{ تا } ۲۱۶,۱۳۵,۴۶,۱۲۷$$

بنابراین:

$$\text{Network} = ۲۱۶,۱۳۵,۴۶,۰$$

$$\text{Broadcast} = ۲۱۶,۱۳۵,۴۶,۱۲۷$$

$$\text{Firsthost} = ۲۱۶,۱۳۵,۴۶,۱$$

$$\text{Lasthost} = ۲۱۶,۱۳۵,۴۶,۱۲۶$$

(ج) مشخصات زیر شبکه: broadcast B، اولین و آخرین host

زیر شبکه:

$$B = 216,135,46,128/26$$

در 26/ اندازه بلوک ۶۴ است، پس بازه آدرس ها:

$$216,135,46,128 \text{ تا } 216,135,46,191$$

بنابراین:

$$\text{Network} = 216,135,46,128$$

$$\text{Broadcast} = 216,135,46,191$$

$$\text{Firsthost} = 216,135,46,129$$

$$\text{Lasthost} = 216,135,46,190$$

(د) مشخصات زیر شبکه: broadcast C، اولین و آخرین host

زیر شبکه:

$$C = 216,135,46,192/27$$

در 27/ اندازه بلوک ۳۲ است، پس بازه آدرس ها:

$$216,135,46,192 \text{ تا } 216,135,46,223$$

بنابراین:

$$\text{Network} = 216,135,46,192$$

$$\text{Broadcast} = 216,135,46,223$$

$$\text{Firsthost} = 216,135,46,193$$

$$\text{Lasthost} = 216,135,46,222$$

سؤال ۴: (NAT) Network Address Translation

سه میزبان داخلی با آدرس های خصوصی 10.0.1.14، 10.0.1.15 و 10.0.1.18 پشت یک روتر NAT قرار دارند. روتر در سمت LAN آدرس 10.0.1.26 و در سمت WAN آدرس عمومی 135.122.203.205 را دارد. میزبان 10.0.1.14 یک دیتاگرام به مقصد 128.119.171.185 با source port برابر 3364 و destination port برابر 80 ارسال می کند.

۱.۴) پاسخ های مفهومی

(الف) چرا روتر NAT باید آدرس مبدأ را هنگام خروج بسته از شبکه محلی تغییر دهد؟

چون آدرس های خصوصی (مثل 10.0.0.0/8) در اینترنت routable نیستند و روترهای اینترنت این آدرس ها را مسیریابی نمی کنند. بنابراین اگر بسته با آدرس مبدأ خصوصی از شبکه خارج شود:

□ در اینترنت قابل مسیریابی نیست و پاسخ هم به آن بر نمی گردد.

□ حتی اگر تقریباً عبور کند، مقصد نمی تواند به یک آدرس خصوصی پاسخ معتبر برگرداند.

پس NAT آدرس مبدأ خصوصی را به آدرس عمومی سمت WAN روتر تبدیل می کند تا ارتباط با اینترنت ممکن شود. علاوه بر این، NAT (به ویژه NAT/PAT) اجازه می دهد چند میزبان داخلی همزمان با یک آدرس عمومی به اینترنت دسترسی داشته باشند.

(ب) روتر NAT چگونه با استفاده از جدول ترجمه، بسته های برگشتی را به میزبان صحیح تحویل می دهد؟

روتر هنگام خروج اولین بسته، یک نگاشت در translation table می سازد تا بداند هر اتصال داخلی با کدام مشخصات خارجی متناظر است. در حالت رایج PAT، این نگاشت معمولاً بر اساس «آدرس و پورت» است (نه فقط آدرس)، یعنی روتر چیزی شبیه این را نگه می دارد:

□ زوج داخلی: (LAN-side IP, LAN-side port)

□ زوج خارجی: (WAN-side IP, WAN-side port)

وقتی بسته برگشتی از اینترنت رسید، روتر به destination IP و destination port نگاه می کند (که همان WAN-side روتر است) و با جستجو در جدول ترجمه، بسته را به LAN-side IP و LAN-side port متناظر بازنویسی کرده و به میزبان داخلی صحیح تحویل می دهد.

۲.۴) تعیین Source و Destination در چهار مرحله

برای هر مرحله، آدرس ها را به صورت یک «چهارتایی» (آی پی و پورت مبدأ/مقصد) می نویسیم. در روابط زیر، از نماد p_{ext} برای پورت خارجی انتخاب شده توسط NAT استفاده می کنیم.

(الف) مرحله ۱: پس از ترک میزبان و قبل از رسیدن به روتر در این مرحله هنوز هیچ ترجمه‌ای انجام نشده است، پس:

$$(srcIP, srcPort) = (10, 0, 1, 14, 3364), \quad (dstIP, dstPort) = (128, 119, 171, 185, 80).$$

(ب) مرحله ۲: پس از ارسال بسته توسط روتر به اینترنت روتر آدرس مبدأ را به آدرس WAN خودش تبدیل می‌کند:

$$(srcIP, srcPort) = (135, 122, 203, 205, p_{ext}), \quad (dstIP, dstPort) = (128, 119, 171, 185, 80).$$

آیا پورت مبدأ تغییر می‌کند؟

□ اگر روتر فقط basic NAT (ترجمه آدرس یک‌به‌یک) انجام دهد، معمولاً پورت تغییر نمی‌کند و می‌توان داشت $p_{ext} = 3364$

□ در حالت رایج‌تر NAT/PAT (اشتراک یک آدرس عمومی بین چند میزبان)، روتر ممکن است پورت را تغییر دهد تا برخوردی بین اتصال‌های مختلف ایجاد نشود؛ در این صورت معمولاً $p_{ext} \neq 3364$ است (اما اگر پورت آزاد باشد، حتی در PAT هم ممکن است همان ۳۳۶۴ انتخاب شود).

پس پاسخ دقیق: «پورت تغییر کند؛ و در PAT معمولاً برای جلوگیری از تداخل تغییر داده می‌شود.»

(ج) مرحله ۳: پس از بازگشت بسته از اینترنت و قبل از ورود به روتر

پاسخ سرور از پورت 80 برمی‌گردد و مقصد آن آدرس عمومی روتر (و پورت خارجی متناظر) است:

$$(srcIP, srcPort) = (128, 119, 171, 185, 80), \quad (dstIP, dstPort) = (135, 122, 203, 205, p_{ext}).$$

(د) مرحله ۴: پس از ارسال بسته توسط روتر به میزبان داخلی

روتر با نگاه کردن به جدول ترجمه، مقصد را از زوج خارجی به زوج داخلی بازنویسی می‌کند:

$$(srcIP, srcPort) = (128, 119, 171, 185, 80), \quad (dstIP, dstPort) = (10, 0, 1, 14, 3364).$$

آیا ورودی جدیدی در جدول NAT ساخته شده است؟

□ اگر این اولین بسته این ارتباط باشد و قبلاً نگاشتی وجود نداشته باشد، روتر هنگام مرحله ۲ یک ورودی جدید ایجاد می کند که به صورت مفهومی چنین است:

$$(135, 122, 203, 205, p_{ext}) \leftrightarrow (10, 0, 1, 14, 3364).$$

□ اما اگر مطابق شکل، از قبل ورودی متناظر وجود داشته باشد، ورودی جدید ساخته نمی شود و همان ورودی موجود استفاده می گردد.

سؤال ۵: تونل زنی و کپسوله سازی IPv6 روی زیرساخت IPv4

مسیر ارسال به صورت زیر داده شده است:

$$D \rightarrow F \rightarrow b \rightarrow a \rightarrow C \rightarrow A$$

در این توپولوژی، روترهای میانی بخش $F \rightarrow b \rightarrow a \rightarrow C$ فقط IPv4 هستند (یا دست کم روی آن بخش از مسیر IPv6 را پشتیبانی نمی کنند) و روترهای F و C دو قابلیت (dual-stack) هستند؛ بنابراین یک تونل IPv6-over-IPv4 بین F و C شکل می گیرد.

۱.۵) پاسخ های تشریحی

(الف) چرا در بخشی از مسیر، دیتاگرام IPv6 باید داخل یک دیتاگرام IPv4 قرار گیرد؟

چون بخشی از مسیر از روترها/زیرساختی عبور می کند که فقط IPv4 را مسیریابی می کنند و قادر به پردازش هدر IPv6 نیستند. اگر بسته به صورت IPv6 خام وارد این بخش شود، روترهای IPv4-only نمی توانند آن را forward کنند و بسته drop می شود. راه حل این است که روتر دو قابلیت ورودی تونل (در اینجا F) دیتاگرام IPv6 را به عنوان payload داخل یک دیتاگرام IPv4 قرار دهد تا برای روترهای میانی «یک بسته IPv4 معمولی» به نظر برسد و بتواند در شبکه IPv4 مسیریابی شود.

(ب) ورودی تونل و خروجی تونل چه نقشی در مسیر ایفا می کنند؟

□ ورودی تونل (tunnel ingress) روتر دو قابلیت ای است که:

□ دیتاگرام IPv6 را دریافت می کند،

- آن را encapsulate می کند (قرار دادن IPv6 داخل IPv4)،
- هدر بیرونی IPv4 را طوری تنظیم می کند که مقصد آن «خروجی تونل» باشد،
- و بسته IPv4 حاصل را به سمت شبکه IPv4 ارسال می کند.
- خروجی تونل (tunnel egress) روتر دو قابلیت‌ای است که:
- بسته IPv4 را دریافت می کند،
- هدر بیرونی IPv4 را حذف می کند (decapsulate)،
- و دیتاگرام IPv6 اصلی را دوباره مثل یک بسته معمولی IPv6 در شبکه IPv6 ادامه مسیر می دهد.
- از دید IPv6، این تونل مثل یک «لینک مجازی» بین دو روتر دو قابلیت‌ای عمل می کند.

۲.۵) تعیین نوع دیتاگرام و آدرس‌های مبدأ/مقصد در هر بخش

برای جلوگیری از نوشتن فارسی داخل ریاضی، از نمادهای زیر استفاده می کنیم:

H_D : آدرس IPv6 میزبان فرستنده در زیر شبکه H_A : آدرس IPv6 میزبان گیرنده در زیر شبکه

F_4 : روی سمت شبکه IPv4 آدرس IPv4 روتر C_4 : روی سمت شبکه IPv4 آدرس IPv4 روتر

(الف) ارسال از F به b

نوع دیتاگرام: IPv4 (بسته بیرونی تونل) که IPv6 را حمل می کند.
آدرس‌های هدر بیرونی:

$$(\text{srcIP}_4, \text{dstIP}_4) = (F_4, C_4)$$

نکته: در IP مقصد هدر بیرونی «خروجی تونل» است (یعنی C)، نه روتر میانی بعدی (b). روتر b صرفاً next-hop در سطح L2 است.

آیا کپسوله شده است؟ بله، IPv6 داخل IPv4 کپسوله شده است.

(ب) ارسال از b به a

نوع دیتاگرام: همچنان IPv4 (درون تونل). روترهای IPv4-only مثل b و a فقط هدر بیرونی IPv4 را می بینند.
آدرس‌های هدر بیرونی:

$$(\text{srcIP}_4, \text{dstIP}_4) = (F_4, C_4)$$

(در طول عبور از روترهای میانی، این دو آدرس ثابت می ماند؛ فقط مقادیری مثل TTL بیرونی کم می شود).
آدرس های هدر درونی (دیتاگرام کپسوله شده):

$$(\text{srcIP}_\epsilon, \text{dstIP}_\epsilon) = (H_D, H_A)$$

یعنی بسته IPv6 داخلی از ابتدا تا انتهای تونل همان مبدأ/مقصد IPv6 را حفظ می کند.

(ج) ارسال از C به A

در روتر C عمل decapsulation انجام می شود، بنابراین:

نوع دیتاگرام: IPv6

آدرس ها:

$$(\text{srcIP}_\epsilon, \text{dstIP}_\epsilon) = (H_D, H_A)$$

آیا کپسوله شده است؟ خیر؛ چون در C هدر بیرونی IPv4 حذف شده و بسته به شکل IPv6 ادامه می دهد.

(د) ورودی تونل، خروجی تونل، و اینکه کدام پروتکل داخل دیگری قرار می گیرد

□ ورودی تونل: روتر F

□ خروجی تونل: روتر C

□ کپسوله سازی: IPv6 داخل IPv4 قرار می گیرد (IPv6-in-IPv4).

سؤال ۶: جداول فلو در OpenFlow

چهار روتر $s1, s2, s3, s4$ داریم و زیرشبکه های متصل به هر روتر به صورت زیر است:

□ $s1$ به $128.119.0.0/16$

□ $s2$ به $128.120.0.0/16$

□ $s3$ به $128.121.0.0/16$

□ $s4$ به $128.122.0.0/16$

هدف پیاده سازی این رفتار است:

□ بسته های TCP از شبکه متصل به s2 به مقصد شبکه متصل به s3 باید از مسیر $s2 \rightarrow s1 \rightarrow s3$ عبور کنند.

□ بسته های UDP با همین مبدأ و مقصد باید مستقیم از مسیر $s2 \rightarrow s3$ عبور کنند.

فرض: هر روتر یک جدول match+action دارد و هر بسته ای که با هیچ قانونی match نشود drop می شود.

۱.۶) پاسخ های تشریحی

(الف) قانون match+action چیست و چرا باید قوانین دقیق تعریف شوند؟

در OpenFlow هر ورودی جدول یک قانون از نوع match+action است:

□ بخش match: روی فیلدهای هدر (مثل IP Src, IP Dst, IP Proto, Src Port, Dst Port) شرط می گذارد (با امکان wildcard).

□ بخش action: کاری که باید روی بسته انجام شود را مشخص می کند (مثلاً output به یک پورت مشخص، یا drop، یا تغییر هدر).

در این سناریو «دقیق بودن» قوانین ضروری است چون:

□ برای یک مبدأ/مقصد یکسان، رفتار TCP و UDP متفاوت است؛ پس باید حداقل روی فیلد IP Proto تفکیک انجام شود.

□ چون default برابر drop است، اگر قانون ها بیش از حد کلی یا اشتباه باشند، یا ترافیک ناخواسته عبور می کند یا ترافیک درست حذف می شود.

□ اگر چند قانون هم پوشان باشند، باید با priority مناسب جلوی انتخاب قانون غلط گرفته شود؛ هر چه match دقیق تر باشد احتمال ابهام و هم پوشانی کمتر می شود.

(ب) اگر بسته ای از شبکه متصل به s4 به مقصد شبکه متصل به s3 ارسال شود چه می شود؟ چه قوانین اضافی لازم است؟

با قوانینی که در این سؤال برای پیاده سازی رفتار خاص $s2 \rightarrow s3$ می سازیم، ترافیک $s4 \rightarrow s3$ معمولاً match نمی شود؛ بنابراین بسته در اولین روتری که قانونی برایش ندارد drop می شود (در عمل از همان s4).
برای پشتیبانی از سناریوی s4 به s3 حداقل این قوانین اضافی لازم است:

□ در s4: قانونی که بسته های با IP Dst=128.121.0.0/16 را به سمت s3 بفرستد (طبق شکل، خروجی پورت ۳).

□ در s3: قانونی که بسته‌هایی با IP Dst=128.121.0.0/16 و IP Src=128.122.0.0/16 را به شبکه محلی خودش تحویل دهد (خروجی پورت ۱).

(اگر هدف «ارتباط کامل همه زیرشبکه‌ها» باشد باید برای همه جفت‌های مبدأ/مقصد نیز قوانین مسیریابی/تحویل محلی اضافه شوند.)

۲.۴) قوانین لازم به صورت match+action

نقشه پورت‌ها از روی شکل

□ s1: پورت ۱ به 128.119/16، پورت ۲ به s2، پورت ۳ به s3 (لینک مورب)، پورت ۴ به s4

□ s2: پورت ۱ به 128.120/16، پورت ۲ به s3، پورت ۳ به s1

□ s3: پورت ۱ به 128.121/16، پورت ۴ به s2، پورت ۳ به s1، پورت ۲ به s4

در جدول‌های زیر، برای فیلدهای پورت لایه ۴ که اهمیتی ندارند از * (یعنی wildcard) استفاده می‌کنیم. همچنین مقدار IP Proto برای TCP برابر ۶ و برای UDP برابر ۱۷ است.

(الف) جدول فلو روتر s1 (فقط برای بسته‌های TCP)

روتر s1 باید بسته TCP مربوط به s2→s3 را به سمت s3 بفرستد (پورت ۳).

Action	IP Proto	Dst Port	Src Port	IP Dst	IP Src
OUTPUT:3	۶	*	*	128.121.0.0/16	128.120.0.0/16

(ب) دو قانون موردنیاز در روتر s2 (یکی برای TCP و یکی برای UDP)

□ TCP: باید به s1 برود (پورت ۳)

□ UDP: باید مستقیم به s3 برود (پورت ۲)

Action	IP Proto	Dst Port	Src Port	IP Dst	IP Src
OUTPUT:3	۶	*	*	128.121.0.0/16	128.120.0.0/16
OUTPUT:2	۱۷	*	*	128.121.0.0/16	128.120.0.0/16

(ج) دو قانون موردنیاز در روتر s3 برای تحویل بسته های TCP و UDP به شبکه محلی روتر s3 باید هر دو نوع بسته رسیده از 128.120/16 به مقصد 128.121/16 را به شبکه محلی خودش بفرستد (پورت ۱).

Action	IP Proto	Dst Port	Src Port	IP Dst	IP Src
OUTPUT:1	۶	*	*	128.121.0.0/16	128.120.0.0/16
OUTPUT:1	۱۷	*	*	128.121.0.0/16	128.120.0.0/16

چک نهایی مسیرها

□ TCP از 128.120/16 به 128.121/16: s3 (OUTPUT:1) → s1 (OUTPUT:3) → s2 (OUTPUT:3)

□ UDP از 128.120/16 به 128.121/16: s3 (OUTPUT:1) → s2 (OUTPUT:2)

سؤال ۷: IP Fragmentation

می خواهیم یک دیتاگرام IPv4 با اندازه 2400 bytes را روی لینکی با MTU=700 bytes ارسال کنیم. شماره شناسه دیتاگرام اولیه 422 است. فرض می کنیم هدر IPv4 بدون options است؛ بنابراین طول هدر برابر 20 bytes است.

(۱) محاسبه تعداد قطعه ها

طول کل دیتاگرام:

$$L = 2400$$

طول هدر:

$$H = 20$$

طول داده قابل حمل در دیتاگرام اصلی:

$$L_{data} = L - H = 2380$$

در هر قطعه، حداکثر طول کل نباید از MTU بیشتر شود:

$$M = 700$$

پس بیشترین داده قابل حمل در هر قطعه:

$$M_{\text{data}} = M - H = 680$$

در IPv4 (به جز قطعه آخر)، اندازه داده هر قطعه باید مضربی از 8 bytes باشد و چون

$$680 \equiv 0 \pmod{8}$$

می توانیم برای قطعه های غیر آخر دقیقاً 680 bytes داده بگذاریم.

تعداد قطعه های کامل:

$$n = \left\lfloor \frac{2380}{680} \right\rfloor = 3$$

باقیمانده داده برای قطعه آخر:

$$r = 2380 - 3 \times 680 = 340$$

پس تعداد کل قطعه ها:

$$N = n + 1 = 4$$

۲) فیلدهای مربوط به قطعه سازی در هر قطعه

فیلدهای کلیدی مرتبط با قطعه سازی:

Identification: برای همه قطعه ها برابر 422 □

Flags: بیت DF باید 0 باشد (چون قطعه سازی انجام می شود) و بیت MF برای همه قطعه ها به جز آخری 1 است. □

Fragment Offset: بر حسب واحدهای 8 bytes □

Total Length: طول کل هر قطعه (هدر + داده) □

از آنجا که Fragment Offset بر حسب 8 bytes است، برای قطعه k ام (با داده 680 bytes در قطعه های اول تا سوم) داریم:

$$\Delta = \frac{680}{8} = 85$$

پس offsetها:

$$O_1 = 0, \quad O_2 = 85, \quad O_3 = 170, \quad O_4 = 255$$

(چون قبل از قطعه چهارم، مقدار داده ارسال شده $2040 = 680 \times 3$ بوده و $255 = 2040/8$).
طول کل هر قطعه:

$$L_1 = L_2 = L_3 = H + 680 = 700, \quad L_4 = H + 340 = 360$$

۳) جدول نهایی قطعه‌ها

Fragment Offset	DF,MF	Identification	Data Length	Total Length	Fragment
۰	۱,۰	۴۲۲	۶۸۰	۷۰۰	۱
۸۵	۱,۰	۴۲۲	۶۸۰	۷۰۰	۲
۱۷۰	۱,۰	۴۲۲	۶۸۰	۷۰۰	۳
۲۵۵	۰,۰	۴۲۲	۳۴۰	۳۶۰	۴

نتیجه

این دیتاگرام به 4 قطعه تقسیم می‌شود. مقدار Identification در همه قطعه‌ها 422 است، MF برای سه قطعه اول 1 و برای قطعه آخر 0 است، و Fragment Offset ها به ترتیب 0, 85, 170, 255 هستند.

سؤال ۸: تأثیر NAT بر برنامه‌های P2P

فرض کنید آلیس پس از lookup متوجه می‌شود باب فایل موردنظر را دارد. هر دو پشت NAT هستند. می‌خواهیم روشی طراحی کنیم که آلیس و باب بدون پیکربندی خاص روی NAT بتوانند یک اتصال TCP با هم برقرار کنند.

ایده اصلی: NAT Traversal با کمک یک سرور عمومی

وقتی هر دو طرف پشت NAT هستند، معمولاً هیچ‌کدام مستقیماً «قابل دسترسی از اینترنت» نیستند؛ چون NAT برای ورودی‌های ناخواسته یک نگاشت (mapping) ندارد و بسته‌های ورودی را drop می‌کند. بنابراین یک راه استاندارد این است که یک موجودیت عمومی (سرور) به‌عنوان هماهنگ‌کننده وجود داشته باشد تا:

□ هر طرف یک اتصال خروجی به سرور بزند (که NAT اجازه می‌دهد)

□ آدرس/پورت عمومی مشاهده‌شده (public endpoint) را به طرف مقابل اعلام کند

□ دو طرف همزمان تلاش کنند اتصال TCP را به هم برقرار کنند (TCP hole punching)
این ایده در عمل معادل استفاده از مفاهیم STUN-like rendezvous و TCP hole punching است.

روش پیشنهادی (گام به گام)

۱) وجود یک Rendezvous Server عمومی

یک سرور عمومی با آدرس عمومی ثابت (روی اینترنت) داریم که هر دو طرف بتوانند به آن TCP connect بزنند. این سرور:

□ ثبت می کند که «کدام کاربر» با «کدام public IP:port» به آن وصل شده است (این public endpoint از دید سرور قابل مشاهده است).

□ وقتی آلیس درخواست باب را می دهد، public endpoint باب را به آلیس و public endpoint آلیس را به باب می دهد.

۲) هر دو طرف همزمان TCP connect می زنند (Simultaneous Open)

بعد از تبادل public endpoint ها، سرور به هر دو پیام می دهد که در یک لحظه تقریباً همزمان:

□ آلیس از پورت محلی مشخص (همان پورتی که به سرور متصل بود یا پورتی که برنامه نگه داشته) به public IP:port باب connect بزند.

□ باب نیز همزمان به public IP:port آلیس connect بزند.

منطق این کار:

□ هر طرف با ارسال یک SYN خروجی، باعث می شود NAT یک نگاشت خروجی بسازد و برای مدت کوتاهی اجازه ورود پاسخ ها را روی همان نگاشت بدهد.

□ اگر زمان بندی درست باشد، SYN های دو طرف از NAT ها عبور می کند و یا یکی از دو SYN به طرف مقابل می رسد و اتصال شکل می گیرد، یا حالت simultaneous open رخ می دهد که در TCP قابل پشتیبانی است.

۳) نقش Keep-Alive

چون نگاشت های NAT timeout دارند، هر طرف می تواند قبل از تلاش اتصال، با ارسال keep-alive به سرور (یا پیام های دوره ای) نگاشت را زنده نگه دارد تا احتمال موفقیت بالا برود.

چرا این روش ممکن است شکست بخورد؟ (محدودیت های واقعی)

حتی بدون نیاز به تنظیم دستی NAT، این روش همیشه تضمینی نیست؛ دلیل اصلی نوع NAT و سیاست های آن است.

۱) Symmetric NAT

در Symmetric NAT، نگاشت پورت خارجی به مقصد وابسته است؛ یعنی اگر کلاینت به سرور وصل شود یک public port می گیرد، اما اگر به همتا وصل شود ممکن است public port متفاوتی بگیرد. در این حالت endpoint ای که سرور دیده، الزاماً همان چیزی نیست که برای اتصال به همتا استفاده می شود، و hole punching معمولاً شکست می خورد.

۲) فیلتر ورودی سخت گیرانه

برخی firewall/NATها فقط اجازه می دهند از همان آدرس مقصدی که قبلاً بسته به آن ارسال شده، بسته برگشتی وارد شود (address-dependent filtering). اگر دو طرف قبلاً ترافیکی به هم ارسال نکرده باشند، بسته ها drop می شوند.

۳) پیچیدگی TCP نسبت به UDP

UDP hole punching معمولاً ساده تر و موفق تر است چون حالت مند نیست؛ اما TCP دارای 3-way handshake و حالت های مختلف است. TCP hole punching عملی است ولی به هماهنگی زمانی و پشتیبانی پیاده سازی نیاز دارد و نرخ شکست بالاتری نسبت به UDP دارد.

اگر روش بالا جواب ندهد چه راهی باقی می ماند؟

اگر هر دو پشت Symmetric NAT یا restrictive firewall باشند، معمولاً بدون پیکربندی خاص، اتصال مستقیم TCP ممکن نیست. در این حالت راه عملی استاندارد:

□ استفاده از Relay روی یک سرور عمومی (معادل TURN)؛ هر دو طرف به سرور وصل می شوند و سرور ترافیک را بین آن ها forward می کند.

این روش «اتصال مستقیم P2P» نیست، ولی همچنان بدون تنظیم NAT کار می کند و از نظر عملی قابل اتکا است.

سؤال ۹: مدار مجازی و شماره های VC

فیلد شماره VC دو بیتی است، پس مجموعه شماره های ممکن:

$$\{00, 01, 10, 11\}$$

شبکه می خواهد یک مدار مجازی جدید روی چهار لینک A,B,C,D ایجاد کند. در حال حاضر روی هر لینک، دو مدار مجازی در حال عبور است و شماره های استفاده شده مطابق جدول زیر است (هر مدار موجود فقط از یکی از این چهار لینک عبور می کند):

A	B	C	D
۱۱	۱۰	۰۱	۰۰
۰۰	۱۱	۱۰	۰۱

پس شماره های اشغال شده روی هر لینک:

$$A: \{11, 00\}, \quad B: \{10, 11\}, \quad C: \{01, 10\}, \quad D: \{00, 01\}.$$

(الف) اگر همه لینک های مسیر باید شماره VC یکسان داشته باشند

در این حالت، یک شماره واحد v باید روی هر چهار لینک آزاد باشد. یعنی v نباید در هیچ کدام از مجموعه های بالا استفاده شده باشد.

مجموع شماره های استفاده شده در چهار لینک:

$$\{11, 00\} \cup \{10, 11\} \cup \{01, 10\} \cup \{00, 01\} = \{00, 01, 10, 11\}.$$

یعنی هر چهار شماره ممکن دست کم روی یکی از لینک ها اشغال شده اند، پس هیچ شماره ای وجود ندارد که همزمان روی A,B,C,D آزاد باشد.

نتیجه:

هیچ شماره VC مشترکی قابل تخصیص نیست.

(ب) اگر لینک ها بتوانند شماره های VC متفاوت داشته باشند

در این حالت، می توان برای هر لینک یک شماره VC جدا انتخاب کرد، به شرطی که آن شماره روی همان لینک آزاد باشد.

ابتدا شماره‌های آزاد هر لینک را پیدا می‌کنیم:

$A : \{01, 10\}$ (اشغال است $\{00, 11\}$ چون)

$B : \{00, 01\}$ (اشغال است $\{10, 11\}$ چون)

$C : \{00, 11\}$ (اشغال است $\{01, 10\}$ چون)

$D : \{10, 11\}$ (اشغال است $\{00, 01\}$ چون)

هر لینک دقیقاً ۲ گزینه آزاد دارد. چون انتخاب‌ها مستقل‌اند (و جدول‌های هدایت می‌توانند شماره‌ها را تبدیل کنند)، تعداد ترکیب‌های چهارتایی ممکن:

$$2 \times 2 \times 2 \times 2 = 2^4 = 16$$

نتیجه:

ترکیب چهارتایی قابل استفاده است. ۱۶

سؤال ۱۰: معماری سوئیچ داخلی در مسیریاب

فرض کنید دو بسته به طور همزمان به دو پورت ورودی متفاوت یک مسیریاب می‌رسند و هیچ بسته دیگری در مسیریاب وجود ندارد. پرسش‌ها درباره عبور این دو بسته از سازه سوئیچینگ داخلی است.

(الف) سوئیچ داخلی از نوع Shared Bus

در معماری گذرگاه مشترک (Shared Bus) همه پورت‌های ورودی و خروجی روی یک گذرگاه مشترک به هم وصل هستند و در هر لحظه عملاً تنها یک انتقال می‌تواند گذرگاه را اشغال کند؛ یعنی گذرگاه یک منبع مشترک و گلوگاه است.

پس حتی اگر دو بسته مقصدهای خروجی متفاوت داشته باشند، برای عبور از داخل مسیریاب هر دو باید از همان گذرگاه مشترک استفاده کنند و بنابراین:

□ همزمان نمی‌توانند عبور کنند،

□ یکی باید منتظر بماند تا گذرگاه آزاد شود،

□ نتیجه عملی این است که حداکثر نرخ عبور همزمان توسط ظرفیت گذرگاه محدود می‌شود.

نتیجه:

خیر؛ در Shared Bus عبور همزمان ممکن نیست.

(ب) سوئیچ داخلی از نوع Crossbar

در معماری نوار متقاطع (Crossbar) بین هر ورودی و هر خروجی یک مسیر قابل برقراری وجود دارد و سوئیچ می تواند چند اتصال ورودی → خروجی را به صورت موازی برقرار کند، به شرط اینکه:

□ دو ورودی متفاوت باشند،

□ دو خروجی هم متفاوت باشند (یعنی روی یک خروجی رقابت نکنند).

در این قسمت گفته شده دو بسته باید به دو پورت خروجی متفاوت بروند، بنابراین می توان دو اتصال مستقل و غیرمتداخل ساخت و هر دو بسته همزمان از سازه عبور می کنند.

نتیجه:

بله؛ در Crossbar اگر خروجی ها متفاوت باشند، عبور همزمان ممکن است.

(پ) هر دو بسته به یک پورت خروجی (سوئیچ داخلی Crossbar)

اینجا هر دو بسته مقصدشان یک خروجی مشترک است. در Crossbar هر خروجی در هر لحظه معمولاً فقط می تواند به یک ورودی متصل باشد (چون پورت خروجی خودش یک منبع واحد است و نمی تواند همزمان دو بسته را دریافت/ارسال کند).

بنابراین حتی اگر سوئیچ قابلیت اتصال های موازی داشته باشد، تعارض روی خروجی باعث می شود:

□ فقط یکی از دو اتصال ورودی → آن خروجی برقرار شود،

□ بسته دیگر در صف ورودی (یا صف خروجی، بسته به طراحی) منتظر بماند،

□ و در اسلات/زمان بعدی عبور کند.

نتیجه:

خیر؛ چون هر دو به یک خروجی می روند، همزمان عبور نمی کنند.

گزارش انجام تمرین ۱۱: پیاده سازی مسیریابی Distance Vector به صورت ناهمگام

(۱) هدف و صورت مسئله

در این تمرین من باید الگوریتم مسیریابی بردار فاصله Distance Vector را به صورت ناهمگام (در یک محیط شبیه سازی شده) پیاده سازی می کردم. شبکه شامل ۴ گره (۰ تا ۳) است و هزینه یال ها مطابق شکل مسئله به صورت زیر است:

□ بین گره های ۰ و ۱ با هزینه ۱

□ بین گره های ۱ و ۲ با هزینه ۱

□ بین گره های ۲ و ۳ با هزینه ۲

□ بین گره های ۰ و ۲ با هزینه ۳

□ بین گره های ۰ و ۳ با هزینه ۷

هدف این بود که هر گره با تبادل بسته های مسیریابی (بردارهای فاصله) در نهایت به کم هزینه ترین مسیرها به همه مقاصد همگرا شود. همچنین در شبیه ساز، یک تغییر هزینه لینک بین گره های ۰ و ۱ در زمان های مشخص رخ می دهد که باید باعث بازهمگرایی درست شبکه شود.

(۲) ایده الگوریتم Distance Vector

در الگوریتم Distance Vector هر گره i یک بردار هزینه کمینه به همه مقصدها نگه می دارد. اگر بردار همسایه k به گره i برسد، گره i برای هر مقصد d مقدار زیر را محاسبه می کند و در جدول خود قرار می دهد:

$$D_i(d | k) = c(i, k) + D_k(d)$$

سپس برای هر مقصد d مقدار کمینه بین همه همسایه ها را به عنوان هزینه نهایی خودش انتخاب می کند:

$$D_i(d) = \min_k D_i(d | k)$$

اگر بردار کمینه گره تغییر کند، گره مجدداً بردار خود را برای همسایه ها ارسال می کند (رویکرد Triggered Update).

۳) ساختار داده ها و پیاده سازی من

من مطابق چارچوب شبیه ساز، برای هر گره دو تابع اصلی را پیاده سازی کردم:

□ `rtinitX()` برای مقداردهی اولیه جدول و ارسال بردار اولیه

□ `rtupdateX(pkt)` برای پردازش بسته دریافتی از همسایه و به روزرسانی جدول

۱.۳) جدول فاصله

برای هر گره X یک ساختار جدول فاصله داشتیم که به صورت 2D نگهداری می شد:

$$dt_X[d][v]$$

که در آن:

□ d مقصد است (۰ تا ۳)

□ v همسایه/مسیر via است (۰ تا ۳)

□ مقدار ذخیره شده یعنی هزینه رسیدن به مقصد d از مسیر همسایه v

ابتدا همه مقادیر را برابر عدد بزرگ (در شبیه ساز ۹۹۹) گذاشتیم تا نقش بی نهایت را بازی کند.

۲.۳) آرایه هزینه لینک های مستقیم

برای هر گره X یک آرایه `connectcostsX` نگه داشتیم که هزینه لینک مستقیم به هر گره را مشخص می کند. برای مثال برای گره ۰:

$$[0, 1, 3, 7]$$

و مشابه آن برای گره های دیگر (طبق توپولوژی).

۳.۳) محاسبه بردار کمینه و تشخیص تغییر

من یک تابع کمکی نوشتیم که برای هر مقصد d کمینه روی ستون های `via` را محاسبه می کند:

$$\min_v dt_X[d][v]$$

سپس با بردار قبلی مقایسه می کنم. فقط اگر تغییر رخ داده باشد، بسته های مسیریابی جدید برای همسایه ها ارسال می شود. این کار باعث می شود ارسال های غیرضروری کم شود و همگرایی تمیزتر باشد.

۴) توضیح دقیق توابع پیاده سازی شده

۱.۴) تابع $rtinitX()$

در هر گره من این مراحل را انجام دادم:

□ همه خانه های جدول را برابر ۹۹۹ قرار دادم.

□ مقدار $dt_X[X][X]$ را برابر ۰ گذاشتم.

□ برای هر همسایه n که لینک مستقیم داشت، مقدار $dt_X[n][n]$ را برابر هزینه لینک مستقیم گذاشتم.

□ بردار کمینه را از روی جدول محاسبه کردم.

□ بردار کمینه را با استفاده از $creatertpkt$ ساخته و با $tolayer2$ برای تمام همسایه ها ارسال کردم.

۲.۴) تابع $rtupdateX(pkt)$

هر زمان بسته ای از همسایه s می رسد:

□ ابتدا بررسی کردم که s واقعاً همسایه است (یعنی لینک مستقیم دارد).

□ برای هر مقصد d مقدار جدید ستون مربوط به s را به صورت زیر به روزرسانی کردم:

$$dt_X[d][s] \leftarrow c(X, s) + pkt.mincost[d]$$

□ سپس بردار کمینه جدید را محاسبه کردم.

□ اگر حداقل یکی از مؤلفه های بردار کمینه عوض شده بود، بردار جدید را برای همسایه ها ارسال کردم.

۵) بخش امتیازی که انجام دادم

من بخش امتیازی مربوط به تغییر هزینه لینک را نیز پیاده سازی کردم؛ یعنی توابع $linkhandler0$ و $linkhandler1$ را نوشتم تا وقتی هزینه لینک بین گره های ۰ و ۱ در شبیه ساز تغییر می کند:

□ مقدار هزینه لینک مستقیم در آرایه $connectcosts$ به روزرسانی شود،

□ ستون مربوط به همان همسایه در جدول فاصله اصلاح شود،

□ و در صورت تغییر بردار کمینه، Triggered Update انجام شود تا شبکه بازهمگرا شود.

در خروجی اجرای من نیز دقیقاً رویدادهای تغییر لینک در زمانهای $t=10000$ و $t=20000$ دیده می شود و شبکه پس از هر تغییر به درستی همگرا می شود.

۶) نتایج اجرا و تحلیل همگرایی

۱.۶) همگرایی اولیه

پس از شروع شبیه ساز (با $TRACE=2$) هر گره جدول اولیه خود را چاپ کرده و بردار اولیه را برای همسایه ها ارسال می کند. سپس با دریافت پیام های $rtupdate$ شبکه مرحله به مرحله به کوتاه ترین مسیرها همگرا می شود. بردارهای نهایی مورد انتظار (کوتاه ترین هزینه ها) قبل از تغییر لینک بین ۰ و ۱ به صورت زیر است:

$$d_0 = [0, 1, 2, 4], \quad d_1 = [1, 0, 1, 3], \quad d_2 = [2, 1, 0, 2], \quad d_3 = [4, 3, 2, 0]$$

در خروجی من:

□ گره ۰ در زمان حدود $t=3.361$ به بردار $[0, 1, 2, 4]$ می رسد و آن را ارسال می کند.

□ گره ۳ در زمان حدود $t=6.315$ به بردار $[4, 3, 2, 0]$ می رسد و آن را ارسال می کند.

این نشان می دهد همگرایی اولیه درست انجام شده است.

۲.۶) تغییر هزینه لینک (بازهمگرایی)

در زمان $t=10000$ هزینه لینک بین گره های ۰ و ۱ به مقدار ۲۰ تغییر می کند. در این شرایط کوتاه ترین مسیرها باید تغییر کنند (مثلاً مسیر رسیدن ۰ به ۱ دیگر مستقیم نیست و از مسیر $0 \rightarrow 2 \rightarrow 1$ می رود). بردارهای جدید مورد انتظار بعد از افزایش هزینه:

$$d_0 = [0, 4, 3, 5], \quad d_1 = [4, 0, 1, 3], \quad d_2 = [3, 1, 0, 2], \quad d_3 = [5, 3, 2, 0]$$

در خروجی من دقیقاً مشاهده می شود:

□ $linkhandler0$ جدول را تغییر می دهد و گره ۰ بردار $[0, 4, 3, 5]$ را ارسال می کند.

□ سپس با دریافت پیام ها، گره ۲ و گره ۱ نیز به مقادیر جدید همگرا می شوند.

نکته مهم: چون شبیه سازی ناهمگام است، ممکن است یک گره برای مدت کوتاهی برداری ارسال کند که هنوز کاملاً تثبیت نشده (به علت نرسیدن آخرین پیام همسایه ها). این رفتار در DV طبیعی است و تا وقتی نهایتاً به بردارهای درست برسد، صحیح محسوب می شود. در خروجی من، شبکه بعد از چند پیام به بردارهای درست بالا همگرا می شود.

۳.۶) بازگشت هزینه لینک و همگرایی مجدد

در زمان $t=20000$ هزینه لینک ۰ و ۱ دوباره به ۱ برمی گردد. در خروجی من:

□ linkhandler0 و linkhandler1 اجرا می شوند،

□ و شبکه دوباره به بردارهای اولیه همگرا می شود (همان d_0, d_1, d_2, d_3 قبل از تغییر).

در انتها نیز شبیه ساز با پیام «عدم وجود بسته در محیط» خاتمه پیدا می کند که نشان دهنده رسیدن به حالت پایدار است.

۳) نتایج اجرا: قرار دادن خروجی و تحلیل مرحله به مرحله

من برنامه را با $TRACE=2$ اجرا کردم. در این بخش، قسمت های کلیدی خروجی را (به صورت گزیده) قرار می دهم و سپس تحلیل می کنم.

۱.۳) همگرایی اولیه (قبل از تغییر لینک)

در ابتدا همه گره ها جدول اولیه را چاپ کرده و بردار اولیه را به همسایه ها می فرستند. به مرور با دریافت پیام ها، بردارها به کوتاه ترین مسیرها همگرا می شوند. برای این توپولوژی، کوتاه ترین هزینه های مورد انتظار (قبل از تغییر لینک ۰ □ ۱) برابر است با:

$$d_0 = [0, 1, 2, 4], \quad d_1 = [1, 0, 1, 3], \quad d_2 = [2, 1, 0, 2], \quad d_3 = [4, 3, 2, 0]$$

گزیده ای از خروجی من که رسیدن به این حالت پایدار را نشان می دهد:

$t=3.361$ rtupdate0: received [1, 0, 1, 3] from node 1 and mincosts updated

$t=3.361$ node0: sending [0, 1, 2, 4] to neighbors.

...

$t=6.315$ rtupdate3: received [2, 1, 0, 2] from node 2 and mincosts updated

$t=6.315$ node3: sending [4, 3, 2, 0] to neighbors.

تحلیل و اثبات درستی همگرایی اولیه من درستی بردارهای نهایی را با محاسبه کوتاه ترین مسیرها بررسی کردم:

□ از ۰ به ۲ مسیر $0 \rightarrow 1 \rightarrow 2$ با هزینه $1 + 1 = 2$ از مسیر مستقیم ۳ ارزان تر است، پس مؤلفه مقصد ۲ در d_0 باید ۲ باشد.

□ از ۰ به ۳ مسیر $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ با هزینه $1 + 1 + 2 = 4$ از مسیر مستقیم ۷ ارزان تر است، پس مؤلفه مقصد ۳ در d_0 باید ۴ باشد.

□ از ۱ به ۳ مسیر $1 \rightarrow 2 \rightarrow 3$ با هزینه $1 + 2 = 3$ است، پس مؤلفه مقصد ۳ در d_1 باید ۳ باشد.

این دقیقاً با بردارهای پایدار مشاهده شده در خروجی (ارسال $[0, 1, 2, 4]$ توسط گره ۰ و ارسال $[4, 3, 2, 0]$ توسط گره ۳) منطبق است.

۲.۳) تغییر هزینه لینک: افزایش هزینه ۱□۰ به ۲۰ در $t=10000$

در زمان $t=10000$ شبیه ساز رویداد تغییر هزینه را اعمال می کند و $linkhandler0$ و $linkhandler1$ فراخوانی می شوند. گزیده خروجی:

```
t=10000.000 linkhandler0: cost to link 1 changed to 20. mincosts updated
t=10000.000 node0: sending [0, 4, 3, 5] to neighbors.
t=10000.000 linkhandler1: cost to link 0 changed to 20. mincosts updated
t=10000.000 node1: sending [3, 0, 1, 3] to neighbors.
```

بعد از چند پیام، شبکه دوباره پایدار می شود و مثلاً گره ۱ هم بردار درست را ارسال می کند:

```
t=10002.132 rtupdate1: received [3, 1, 0, 2] from node 2 and mincosts updated
t=10002.132 node1: sending [4, 0, 1, 3] to neighbors.
```

بردارهای مورد انتظار بعد از افزایش هزینه ۱□۰ برابر ۲۰ می شود، کوتاه ترین هزینه ها باید به صورت زیر تغییر کند:

$$d_0 = [0, 4, 3, 5], \quad d_1 = [4, 0, 1, 3], \quad d_2 = [3, 1, 0, 2], \quad d_3 = [5, 3, 2, 0]$$

تحلیل و اثبات درستی بعد از تغییر من درستی این بردارها را با استدلال کوتاه ترین مسیرها ثابت کردم:

□ برای ۰ به ۱: مسیر مستقیم هزینه ۲۰ دارد، اما مسیر $0 \rightarrow 2 \rightarrow 1$ هزینه $4 + 1 = 5$ دارد، پس مؤلفه مقصد ۱ در d_0 باید ۴ باشد.

□ برای ۰ به ۳: بهترین مسیر $0 \rightarrow 2 \rightarrow 3$ با هزینه $4 + 2 = 6$ است (از مسیر مستقیم ۷ ارزان تر)، پس مؤلفه مقصد ۳ در d_0 باید ۵ باشد.

□ برای ۳ به ۰: مسیر مستقیم ۷ است، ولی مسیر $3 \rightarrow 2 \rightarrow 0$ هزینه $3 + 2 = 5$ دارد، پس $d_3[0] = 5$.

در خروجی من، دقیقاً مشاهده می شود که بلافاصله پس از اجرای $linkhandler0$ گره ۰ بردار $[0, 4, 3, 5]$ را ارسال می کند که با محاسبات کوتاه ترین مسیر کاملاً منطبق است. همچنین گره ۱ ابتدا ممکن است به صورت موقت بردار $[3, 0, 1, 3]$ ارسال کند (به علت ناهمگام بودن و نرسیدن اطلاعات جدید همسایه ها)، اما پس از دریافت بردار صحیح از گره ۲، به بردار پایدار $[4, 0, 1, 3]$ همگرا می شود؛ این دقیقاً رفتار طبیعی DV در حالت ناهمگام است و در نهایت به نقطه ثابت صحیح می رسد.

۳.۳) بازگشت هزینه لینک به ۱ در $t=20000$

در $t=20000$ هزینه لینک ۱ دوباره به ۱ برمی گردد و بازهمگرایی انجام می شود:

$t=20000.000$ linkhandler0: cost to link 1 changed to 1. mincosts updated

$t=20000.000$ node0: sending [0, 1, 2, 4] to neighbors.

$t=20000.000$ linkhandler1: cost to link 0 changed to 1. mincosts updated

$t=20000.000$ node1: sending [1, 0, 1, 3] to neighbors.

این دقیقاً همان بردارهای پایدار اولیه است و نشان می دهد linkhandler به درستی هزینه را برگردانده و Triggered Update درست انجام شده است.

۴) استدلال کلی صحت پیاده سازی

برای اثبات کلی درستی، من به دو نکته تکیه می کنم:

۱.۴) ناوردای جدول فاصله (در هر گره)

در پیاده سازی من، بعد از هر دریافت بسته از همسایه k ، ستون $\text{via}=k$ دقیقاً طبق رابطه زیر ساخته می شود:

$$dt_i[d][k] = c(i, k) + D_k(d)$$

و بردار کمینه هم دقیقاً با کمینه گیری روی ستون ها محاسبه می شود:

$$D_i(d) = \min_k dt_i[d][k]$$

بنابراین من دقیقاً گام Bellman-Ford را پیاده سازی کرده ام.

۲.۴) تطبیق نقطه پایدار با کوتاه ترین مسیرها

الگوریتم Distance Vector در حالت پایدار یک نقطه ثابت از روابط Bellman-Ford است؛ و این نقطه ثابت (با فرض نبود حلقه های منفی) همان کوتاه ترین مسیرها را می دهد. در این تمرین، هزینه ها همگی نامنفی هستند، بنابراین جواب کوتاه ترین مسیر یکتا و پایدار است. سپس من با محاسبات دستی کوتاه ترین مسیرها (قبل از تغییر لینک و بعد از تغییر لینک) نشان دادم بردارهای مشاهده شده در خروجی دقیقاً با کوتاه ترین مسیرها یکسان هستند؛ بنابراین خروجی ها درست هستند.

۵) جمع بندی نهایی

- من بخش اصلی را با پیاده سازی `rtinitX` و `rtupdateX` مطابق DV انجام دادم.
- من بخش امتیازی را هم انجام دادم: `linkhandler0` و `linkhandler1` را طوری پیاده سازی کردم که تغییر هزینه لینک مستقیماً روی جدول فاصله اثر داده شود و سپس Triggered Update انجام شود.
- من درستی نتایج را با دو روش اثبات کردم: (۱) تطبیق دقیق خروجی های پایدار با کوتاه ترین مسیرهای محاسبه شده، (۲) توضیح ناوردای Bellman-Ford در ساختار جدول و منطق به روزرسانی.

مراجع